

Train Model.py

This script:

1. Reads collected sign language images
2. Extracts hand landmarks using MediaPipe
3. Converts landmarks into numerical features
4. Trains a Random Forest classifier
5. Evaluates and saves the trained model

👉 This turns **raw images** → **intelligent model**

◆ Imports Explained

```
import os
import pickle
import numpy as np
import cv2
import mediapipe as mp
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import time
```

Library	Purpose
os	Directory & file handling
pickle	Saving/loading dataset & model
numpy	Numerical arrays
cv2	Image reading
mediapipe	Hand landmark detection
sklearn	ML training, evaluation
time	Measure training time

◆ Configuration Section

```
DATA_DIR = './data/imgs'
```

```
DATASET_DIR = './data/dataset'
```

```
MODEL_DIR = './models'
```

```
MODEL_FILE = 'sign_language_model.pkl'
```

Meaning:

- DATA_DIR → Raw images
- DATASET_DIR → Processed numerical data
- MODEL_DIR → Saved ML model
- MODEL_FILE → Final trained model

✦ Separation improves **modularity & debugging**

◆ MediaPipe Setup

```
mp_hands = mp.solutions.hands
```

```
mp_drawing = mp.solutions.drawing_utils
```

```
mp_drawing_styles = mp.solutions.drawing_styles
```

Why MediaPipe?

- Fast
 - Accurate
 - Pre-trained hand landmark detection
 - No manual feature engineering required
-

◆ Function 1: create_directory(path)

Same as previous script.

- ✓ Prevents runtime errors
 - ✓ Ensures required folders exist
-

◆ Function 2: extract_hand_landmarks(image_path)

MOST IMPORTANT FUNCTION (VIVA FAVORITE)

```
def extract_hand_landmarks(image_path):
```

◆ Step 1: Read Image

```
img = cv2.imread(image_path)
```

- Reads image from disk
 - Returns None if corrupted or missing
-

◆ Step 2: Convert to RGB

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Why?

- OpenCV → BGR
 - MediaPipe → RGB
-

◆ Step 3: MediaPipe Hands Model

```
with mp_hands.Hands(  
    static_image_mode=True,  
    max_num_hands=1,  
    min_detection_confidence=0.3  
) as hands:
```

Parameter	Meaning
static_image_mode=True	Image-by-image processing
max_num_hands=1	Only one hand expected
min_detection_confidence=0.3	Low threshold to capture more hands

◆ Step 4: Landmark Detection

```
results = hands.process(img_rgb)
```

If hand detected:

```
results.multi_hand_landmarks
```

◆ Step 5: Feature Extraction

```
for landmark in hand_landmarks.landmark:
```

```
    landmarks.extend([landmark.x, landmark.y, landmark.z])
```

Output:

- **21 landmarks**
- Each landmark → (x, y, z)
- Total features = **63**

✦ These are **scale-invariant normalized coordinates**

◆ Return Value

Case	Return
------	--------

Hand detected	List of 63 values
---------------	-------------------

No hand	None
---------	------

◆ Function 3: `load_and_process_data()`

Purpose:

Convert images → ML-ready dataset

◆ Step 1: Detect Classes

```
classes = [d for d in os.listdir(DATA_DIR) if os.path.isdir(...)]
```

Each folder = one gesture class

◆ Step 2: Assign Labels

```
labels.append(class_idx)
```

Gesture Label

A	0
---	---

B	1
---	---

Hello	2
-------	---

✦ Label encoding is required for ML models

◆ Step 3: Loop Through Images

```
landmarks = extract_hand_landmarks(img_path)
```

- ✓ Only successful extractions are used
 - ✓ Failed images are skipped safely
-

◆ Step 4: Final Output

```
return np.array(data), np.array(labels), classes
```

Variable Meaning

X Feature matrix ($N \times 63$)

y Labels

classes Class names

◆ Function 4: save_dataset(X, y, classes)

```
pickle.dump(dataset, f)
```

Why save dataset?

- Avoid recomputation
 - Faster retraining
 - Debugging & reproducibility
-

◆ Function 5: train_model(X, y, classes)

CORE ML FUNCTION

◆ Step 1: Train-Test Split

```
train_test_split(..., test_size=0.2, stratify=y)
```

Feature	Reason
test_size=0.2	80–20 split
stratify=y	Maintains class balance
random_state=42	Reproducibility

◆ Step 2: Random Forest Classifier

```
RandomForestClassifier(  
    n_estimators=100,
```

```
max_depth=20,  
min_samples_split=5,  
min_samples_leaf=2  
)
```

Why Random Forest?

- ✓ Handles non-linear data
 - ✓ Robust to noise
 - ✓ No feature scaling needed
 - ✓ Works well on landmark data
-

◆ **Step 3: Model Training**

```
model.fit(X_train, y_train)
```

Time is recorded for performance analysis.

◆ **Step 4: Model Evaluation**

Training Accuracy

```
accuracy_score(y_train, y_train_pred)
```

Testing Accuracy

```
accuracy_score(y_test, y_test_pred)
```

◆ **Step 5: Classification Report**

```
classification_report(...)
```

Includes:

- Precision
 - Recall
 - F1-Score
 - Support
-

◆ **Step 6: Confusion Matrix**

```
confusion_matrix(y_test, y_test_pred)
```

Shows:

- Misclassifications
 - Class confusion patterns
-

◆ Function 6: `save_model(model, classes)`

```
pickle.dump(model_data, f)
```

Saved:

- Trained model
- Class label mapping

✦ Required for **real-time prediction UI**

◆ Function 7: `main()`

Pipeline Controller

1. Load data
2. Save dataset
3. Train model
4. Save model

Ensures **clean ML workflow**

◆ `if __name__ == "__main__":`

Prevents unintended execution when imported in:

- Tkinter UI
- Real-time prediction script

VIVA QUESTIONS & ANSWERS (VERY IMPORTANT)

Q1. Why use MediaPipe instead of raw images?

Answer:

MediaPipe extracts meaningful hand landmarks, reducing input size and improving classification accuracy.

Q2. Why 63 features?

Answer:

MediaPipe detects 21 landmarks per hand, each with x, y, z coordinates → $21 \times 3 = 63$ features.

Q3. Why Random Forest instead of CNN?

Answer:

Because the input is structured landmark data, not raw pixels. Random Forest performs well on tabular data and requires less computation.

Q4. What is `static_image_mode=True`?

Answer:

It processes each image independently, which is ideal for dataset preparation.

Q5. Why do some images fail landmark extraction?

Answer:

Poor lighting, blurred images, occlusion, or incorrect hand orientation.

Q6. What is stratified splitting?

Answer:

It ensures that each class is proportionally represented in both training and testing datasets.

Q7. What does confusion matrix show?

Answer:

It shows how many samples were correctly classified and which classes were confused with others.

Q8. Why save model using pickle?

Answer:

Pickle preserves trained parameters and allows reuse without retraining.

Q9. Can this model work in real time?

Answer:

Yes. By extracting landmarks from live webcam frames and passing them to the trained model.

Q10. How can accuracy be improved?

Answer:

- More training data
- Better lighting
- Data augmentation
- Using CNN or LSTM for temporal gestures

 Examiner-Ready Final Summary

“This module converts raw sign language images into numerical hand landmarks using MediaPipe, trains a Random Forest classifier, evaluates its performance, and stores the trained model for real-time gesture recognition applications.”